PROJECT REPORT

ON

ML NETWORK INTRUSION DETECTOR

A Machine Learning-Based Network Traffic Classification System

Submitted in partial fulfillment of the requirements for

[Course / Subject Name]

Submitted by

[Student Name]

[Roll Number / Registration Number]

Under the guidance of

[Guide / Faculty Name]

[Department Name]

[College / Institution Name]

[University Name, if applicable]

Academic Year: [20__–20__]

CERTIFICATE

This is to certify that the project report entitled “ML Network Intrusion Detector” is prepared by [Student Name], bearing Roll Number [Roll Number], under my guidance as part of the academic requirements of [Course / Department Name]. The work describes an educational prototype developed using Python, Streamlit, Pandas, and Scikit-learn for analyzing labeled network traffic data and classifying records into learned classes.

Project Guide Signature	Head of Department Signature
Name: _____	Name: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide and faculty members for their guidance and support during the development of this project. I also thank my department and institution for providing the opportunity to explore machine learning and cybersecurity through a practical application. Finally, I acknowledge the developers and maintainers of Python, Streamlit, Pandas, and Scikit-learn, whose tools made it possible to build this prototype.

ABSTRACT

Network intrusion detection is an important part of cybersecurity because network traffic may contain patterns associated with unauthorized access, misuse, or other malicious activity. This project, “ML Network Intrusion Detector,” presents an educational web application that uses machine learning to classify records from a labeled network-traffic CSV dataset. The application is developed in Python and uses Streamlit for the user interface, Pandas for data handling, and Scikit-learn to train a Random Forest classifier. Users can upload a dataset, inspect records and missing values, select a target-label column, train and evaluate a model, review a confusion matrix and classification report, and analyze a selected record. Feature importance is also displayed to help users understand which input features influenced the model. The system is intended for learning and demonstration. It analyzes uploaded data and does not monitor live network traffic, block connections, or replace a production intrusion detection system. Model performance depends on dataset quality, label correctness, and the way data is prepared and evaluated.

Keywords: Cybersecurity, Intrusion Detection, Machine Learning, Random Forest, Python, Streamlit, Network Traffic.

TABLE OF CONTENTS

1. Introduction

2. Problem Statement

3. Objectives

4. Scope of the Project

5. Technologies Used

6. System Requirements

7. System Design and Workflow

8. Methodology and Implementation

9. Features and Modules

10. Testing and Evaluation

11. Results and Discussion

12. Limitations

13. Future Enhancements

14. Conclusion

15. References

Appendix A: How to Run the Project

1. INTRODUCTION

Computer networks support communication and information sharing across organizations, institutions, and individuals. These networks can be exposed to threats such as denial-of-service attempts, scanning, credential attacks, and suspicious communication. An intrusion detection system (IDS) examines events or traffic to identify activity that may require investigation.

Machine learning can assist this process by learning patterns from labeled examples. In this project, a Random Forest classifier is trained on a user-provided CSV dataset. A Streamlit interface makes the workflow accessible to users without requiring them to interact directly with Python scripts for each analysis.

2. PROBLEM STATEMENT

Manual inspection of large network-traffic datasets can be time-consuming and difficult for beginners. The project addresses the need for a simple educational tool that can load labeled traffic records, explore their structure, train a basic classification model, and display evaluation results in an understandable dashboard. The tool is a prototype for offline dataset analysis rather than a live operational security system.

3. OBJECTIVES

- Develop a user-friendly cybersecurity dashboard using Streamlit.
- Allow users to upload labeled network traffic data in CSV format.
- Display dataset dimensions, column types, unique values, and missing-value counts.
- Preprocess data and train a Random Forest classification model.
- Evaluate the model using accuracy, a confusion matrix, and a classification report.
- Show feature importance and classify a selected record from the uploaded dataset.
- Explain common network threats and basic security practices.

4. SCOPE OF THE PROJECT

The application supports offline analysis of a labeled CSV file. It is designed for academic demonstration, basic experimentation, and learning about machine-learning-based classification. It can work with different datasets when their columns and target labels are suitable for classification. The user must choose the correct target column and understand the dataset's label meanings. The application does not capture packets, connect to routers, inspect a live network, or automatically respond to incidents.

5. TECHNOLOGIES USED

Technology	Purpose	Use in the Project
Python	Programming language	Implements data processing, training, and application logic.
Streamlit	Web interface	Creates pages, navigation, upload controls, metrics, and charts.
Pandas	Data analysis	Reads CSV files and inspects, transforms, and summarizes tabular data.

Scikit-learn	Machine learning	Splits data, trains Random Forest, and calculates evaluation metrics.
Matplotlib	Plotting support	Available for additional plots and visualizations.

6. SYSTEM REQUIREMENTS

6.1 Hardware Requirements

- A computer capable of running Python and a modern web browser.
- Recommended: 4 GB RAM or more for small-to-medium educational datasets.
- Sufficient storage for the project files and dataset.

6.2 Software Requirements

- Windows, Linux, or macOS.
- Visual Studio Code (VS Code) or another Python editor.
- A compatible Python 3 installation.
- Streamlit, Pandas, Scikit-learn, and Matplotlib installed in the Python environment.
- A labeled CSV network-traffic dataset.

7. SYSTEM DESIGN AND WORKFLOW

The overall workflow is as follows:

1. User opens the Streamlit application.
2. User uploads a labeled network-traffic CSV file.
3. The application previews the data and reports its structure and missing values.
4. User selects the column containing the class label.
5. The application converts categorical input fields into numeric features and handles missing values.
6. The dataset is split into training and test subsets.
7. A Random Forest classifier is trained using the training subset.
8. Predictions on the test subset are compared with known labels.
9. Evaluation metrics and feature-importance information are displayed.
10. The user can select a record for classification and consult the security guide.

Data flow: CSV upload → data inspection → target selection → preprocessing → train/test split → Random Forest training → evaluation → record prediction and visualizations.

8. METHODOLOGY AND IMPLEMENTATION

8.1 Data Input

The application accepts a CSV file through Streamlit's file uploader. The dataset should contain input features describing network activity and a target column containing the known class for each record. The target may use names such as Normal and Attack, but the actual values depend on the chosen dataset.

8.2 Data Preprocessing

Rows without a target value are removed. The target values are converted to text labels. The selected target column is separated from the input features. Categorical input columns are converted into numeric indicator columns using one-hot encoding. Infinite values are replaced with missing values, and remaining missing feature values are filled with zero. The resulting feature columns are aligned during record prediction.

For a rigorous production model, preprocessing should be fitted only on training data, and the dataset should be checked for duplicate records, target leakage, identifiers, and class imbalance. The current implementation is a learning prototype and may require adjustments for particular datasets.

8.3 Model Training

The project uses RandomForestClassifier from Scikit-learn. A Random Forest combines predictions from multiple decision trees to produce a classification result. The dataset is split into training and test subsets, with a configurable test percentage. A fixed random seed is used to make the split and model initialization more reproducible. Class weighting is enabled to help account for unequal class frequencies.

8.4 Evaluation

The application reports test accuracy, a confusion matrix, and a classification report. Accuracy is the fraction of test examples classified correctly. Precision measures the proportion of predicted examples for a class that are correct; recall measures the proportion of actual examples of a class that are found; and the F1-score combines precision and recall. These measures should be reviewed together, particularly when attack examples are much less common than normal examples.

8.5 Record Classification

After training, the user can select a record from the uploaded dataset. The input features are converted and aligned to the model's training columns, then passed to the classifier. The predicted class and model probability estimates are displayed. Probability estimates are not proof that a record is malicious or safe, and the class labels must be interpreted according to the dataset.

9. FEATURES AND MODULES

Dashboard: Shows the project purpose, dataset status, record count, and latest model test accuracy.

Dataset Explorer: Displays data previews, column information, missing values, summaries, and charts.

Train Model: Lets the user select the target column and test percentage, trains Random Forest, and shows evaluation results.

Detect Traffic: Classifies a selected record using the trained model and displays probability estimates.

Security Guide: Provides descriptions of common attack categories, security practices, and evaluation metrics.

10. TESTING AND EVALUATION

The following test cases can be used to verify the application. Actual outcomes should be recorded after running the project with a suitable dataset.

Test Case	Expected Behaviour	Status

Open app without a file	Dashboard loads and prompts for a CSV upload.	To be verified
Upload a valid labeled CSV	Preview and dataset statistics are displayed.	To be verified
Choose target column and train	Model trains and evaluation metrics appear.	To be verified
Choose an invalid/single-class target	Application shows a helpful error.	To be verified
Classify a record after training	Predicted class and probability chart appear.	To be verified
Upload an empty or unreadable file	Application displays an error instead of crashing.	To be verified

11. RESULTS AND DISCUSSION

The application is designed to present model evaluation results after the user uploads a dataset and trains the classifier. Since accuracy and other metrics depend on the actual dataset and the run configuration, no numerical performance result is claimed in this report. After completing a successful run, insert the measured test accuracy, dataset name, number of records, target labels, and screenshots of the Dashboard, Dataset Explorer, model evaluation, and prediction page.

Results to complete after testing:

Dataset name/version: _____

Number of records and input features: _____

Target label column and class names: _____

Test split percentage: _____

Measured test accuracy: _____

Important observations / errors found: _____

12. LIMITATIONS

- The application analyzes uploaded CSV data and does not capture live network traffic.
- Prediction quality depends on the representativeness and correctness of the dataset.
- A high accuracy score alone does not guarantee good detection of rare attacks.
- Dataset-specific columns may require additional preprocessing or removal.
- Model probability estimates can be misinterpreted and should not be treated as certainty.
- The prototype does not provide automated blocking, alert delivery, or incident response.

13. FUTURE ENHANCEMENTS

- Add support for validated public datasets such as NSL-KDD or CICIDS, with dataset-specific preprocessing.
- Use a Scikit-learn Pipeline to fit preprocessing only on the training subset and reduce data leakage.

- Add precision-recall curves, ROC-AUC where appropriate, and class-wise metric visualizations.
- Add cross-validation and hyperparameter tuning.
- Provide model export and reload functionality for repeatable experiments.
- Add real-time packet capture only in a controlled, authorized test environment.
- Introduce user authentication, audit logs, and responsible alert handling for a more advanced deployment.

14. CONCLUSION

The ML Network Intrusion Detector demonstrates how machine learning can be combined with a web interface to explore labeled network traffic and classify records. Built with Python, Streamlit, Pandas, and Scikit-learn, the project brings dataset inspection, Random Forest training, evaluation metrics, feature importance, and record prediction into one application. It provides a practical introduction to data preprocessing and machine-learning evaluation in cybersecurity. The prototype is intended for academic use; further validation, robust preprocessing, and controlled testing would be required before using such a system in a real security environment.

15. REFERENCES

11. Python Software Foundation. Python Documentation. https://docs.python.org/3/
12. Streamlit. Streamlit Documentation. https://docs.streamlit.io/
13. Pandas. Pandas Documentation. https://pandas.pydata.org/docs/
14. Scikit-learn. RandomForestClassifier Documentation. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html
15. Scikit-learn. Model Evaluation Documentation. https://scikit-learn.org/stable/modules/model_evaluation.html
16. Scikit-learn. train_test_split Documentation. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

APPENDIX A: HOW TO RUN THE PROJECT

1. Open the project folder in Visual Studio Code.

2. Open the integrated terminal.

3. (Recommended) Create and activate a virtual environment:

```
python -m venv venv
venv\Scripts\activate
```

4. Install the required packages:

```
pip install streamlit pandas scikit-learn matplotlib
```

5. Save the application code in a file named app.py.

6. Start the application:

```
streamlit run app.py
```

7. Open the local URL shown in the terminal, upload a labeled CSV dataset, explore it, select its target column, train the model, and review the results.